

Change Data Capture

데이터베이스 변경을 감지하고 전파하는 기술

데이터 엔지니어 관점에서 바라본 CDC의 개념, 아키텍처, 그리고 실전 패턴

오늘 다룰 내용

1. SCD Type 2에서 CDC로 — 왜 CDC가 필요한가
2. CDC 핵심 개념 — 어떻게 변경을 감지하는가
3. CDC 아키텍처 — 전체 파이프라인 구조와 일관성 보장
4. CDC의 활용 범위 — 분석 그 너머의 use case
5. 애플리케이션 이벤트와의 차이 — CDC vs. 직접 이벤트 발행
6. Dual Write Problem & Outbox Pattern — 안전한 이벤트 발행
7. 정리 — 언제, 무엇을 쓸 것인가

1. SCD Type 2에서 CDC로

변경 이력을 보존하려면, 먼저 변경을 감지해야 한다

SCD Type 2

변경 이력을 별도 행으로 보존하는 차원 모델링 기법

user_id	email	valid_from	valid_to
42	old@x.com	2024-01-01	2024-06-15
42	new@x.com	2024-06-15	NULL

- `valid_to IS NULL` → 현재 유효한 행
- 특정 시점의 상태를 쿼리로 조회 가능 (Time Travel)

SCD Type 2는 **변경을 어떻게 저장할까**의 문제를 해결한다

그런데 — 변경을 어떻게 감지하지?

SCD Type 2가 저장 전략이라면, 그 전에 해결해야 할 질문이 있다:

"원본 DB에서 데이터가 바뀐 걸 어떻게 알아챌까?"

이 질문에 대한 답이 바로 CDC (Change Data Capture)

관심사	기술	질문
변경 감지	CDC	무엇이 바뀌었는가?
변경 저장	SCD Type 2	바뀐 걸 어떻게 보관할까?

CDC가 감지하고, SCD Type 2가 저장한다 — 실무에서 이 둘은 거의 항상 함께 다뤄진다.

2. CDC 핵심 개념

INSERT, UPDATE, DELETE를 실시간으로 캡처한다

CDC란?

데이터베이스에서 발생하는 INSERT, UPDATE, DELETE 이벤트를 실시간(또는 준실시간)으로 캡처하는 기법

예시

`users` 테이블에서 이메일이 변경되면:

"user_id=42의 email이 `old@x.com` → `new@x.com` 으로 변경되었다"

라는 이벤트를 하류(downstream) 시스템에 전달한다.

변경을 감지하는 방법에는 크게 두 가지 접근법이 있다.

접근법 ① Query-based CDC (폴링)

주기적으로 원본 테이블을 **조회(SELECT)**해서 변경사항을 찾는 방식

```
-- "마지막 체크 이후 바뀐 행"을 가져온다  
SELECT * FROM users  
WHERE updated_at > '2024-06-15 10:00:00';
```

한계점:

- DELETE 를 감지하기 어려움 — 행 자체가 사라지므로 SELECT로 잡을 수 없음
- 폴링 간격 사이의 변경은 놓칠 수 있음
- Source DB에 SELECT * 쿼리 부하가 발생

접근법 ② Log-based CDC

DB 내부의 트랜잭션 로그를 직접 읽는 방식. CDC의 핵심.

DB	트랜잭션 로그	설명
MySQL	binlog (Binary Log)	모든 데이터 변경을 바이너리로 기록
PostgreSQL	WAL (Write-Ahead Log)	커밋 전 변경사항을 먼저 기록
MongoDB	Oplog (Operation Log)	replica set의 모든 연산을 기록

왜 이게 가능한가?

DB에 커밋된 모든 변경은 반드시 트랜잭션 로그에 순서대로 기록된다.

Debezium 같은 CDC 도구가 이 로그를 읽어 이벤트 스트림으로 변환한다.

Log-based CDC의 장점

항목	Query-based	Log-based
DELETE 감지	어려움	완벽히 캡처
Source DB 부하	SELECT 쿼리 부하	거의 없음 (로그만 읽음)
이벤트 순서 보장	보장 안 됨	로그 순서 = 커밋 순서
지연 시간	폴링 간격에 의존	초 ~ 밀리초 단위
before/after 상태	이전 상태 모름	변경 전후 모두 캡처

Debezium은 자신을 DB Replica처럼 등록해서 로그를 수신한다

MySQL → replica로 등록 / PostgreSQL → logical replication slot 생성

CDC 이벤트 구조 — Debezium 기준

```
{
  "op": "u",
  "before": {
    "user_id": 42,
    "email": "old@x.com",
    "name": "Kim"
  },
  "after": {
    "user_id": 42,
    "email": "new@x.com",
    "name": "Kim"
  },
  "source": {
    "db": "mydb",
    "table": "users",
    "ts_ms": 1707300000000,
    "lsn": 12345678
  }
}
```

이벤트 필드 해설

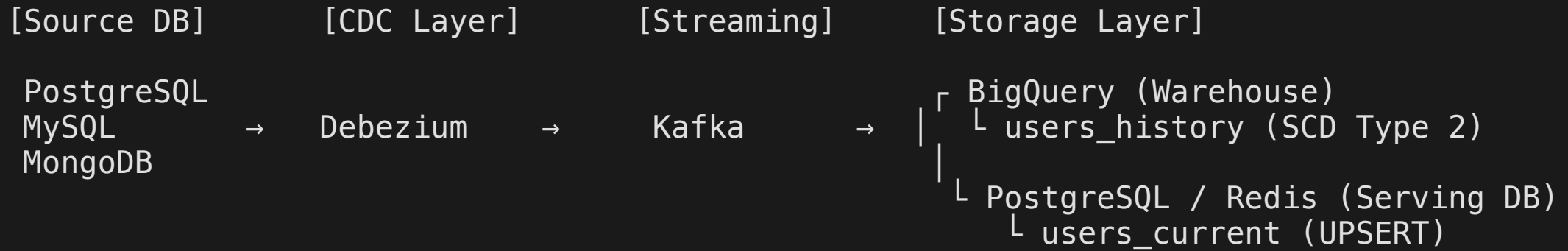
필드	의미	비고
op	변경 종류	c =CREATE, u =UPDATE, d =DELETE, r =snapshot
before	변경 전 행 상태	INSERT이면 null
after	변경 후 행 상태	DELETE이면 null
source.db	원본 데이터베이스명	
source.table	원본 테이블명	
source.ts_ms	변경 발생 시각	epoch milliseconds
source.lsn	Log Sequence Number	이벤트 순서 판별의 핵심

lsn 이 클수록 더 나중에 발생한 이벤트 — 뒤에서 다룰 UPSERT 일관성 보장의 핵심

3. CDC 아키텍처

전체 파이프라인은 어떻게 구성되는가

전체 아키텍처 — Big Picture



계층	기술 예시	역할	쓰기 패턴
Source	PostgreSQL, MySQL	서비스 원본	일반 CRUD
CDC	Debezium + Kafka	변경 감지 + 전달	PK 기반 파티셔닝
Warehouse	BigQuery, Snowflake	이력 분석	SCD Type 2 APPEND
Serving	PostgreSQL, Redis	최신 상태 조회	Conditional UPSERT

Warehouse vs. Serving DB — 왜 나누는가?

Warehouse (BigQuery, Snowflake 등)

- "지난 6개월간 이메일을 변경한 유저 수" 같은 분석 쿼리 용도
- APPEND 위주 → 컬럼 스토리지 엔진과 궁합이 좋음

Serving DB (PostgreSQL, Redis 등)

- API 서버가 "user_id=42의 현재 이메일"을 밀리초 내 조회해야 할 때
- BigQuery를 찌르면 레이턴시가 너무 높음

항상 두 개가 필요한 건 아님 —

분석만 하면 Warehouse 안에서 `_history` + `_current` 테이블로 관리하기도 함

"current" 테이블 유지 패턴

CDC 이벤트를 받을 때 두 곳에 동시에 적재하는 실무 패턴:

```
CDC Event → [Consumer]
├── users_history (SCD Type 2 - APPEND)
│   ├── user_id=42, email=old@x.com, valid_from=..., valid_to=now
│   └── user_id=42, email=new@x.com, valid_from=now, valid_to=NULL
└── users_current (최신 상태 - UPSERT)
    └── user_id=42, email=new@x.com
```

- 이력 조회 → `users_history`
- 최신 조회 → `users_current` (항상 1행/PK)

Materialized View 대신 별도 테이블을 유지하는 것이 실무에서 가장 흔한 패턴

UPSERT 일관성 — 문제 상황

이벤트가 순서대로 도착하지 않으면?

user_id=42에 대해 UPDATE가 두 번 발생:

이벤트 A: email → "a@x.com" (lsn=100) ← 먼저 발생

이벤트 B: email → "b@x.com" (lsn=200) ← 나중에 발생

네트워크 지연으로 B가 먼저 도착하고, A가 나중에 도착하면?

1. B 도착 → UPSERT → email = "b@x.com" (0)

2. A 도착 → UPSERT → email = "a@x.com" (X) 최종 상태가 틀림!

단순 UPSERT는 최종 상태를 보장하지 못한다.

UPSERT 일관성 — 해결: Conditional UPSERT

```
-- PostgreSQL: LSN 기반 Conditional UPSERT
INSERT INTO users_current (user_id, email, _cdc_lsn)
VALUES (42, 'a@x.com', 100)
ON CONFLICT (user_id) DO UPDATE
SET email      = EXCLUDED.email,
    _cdc_lsn   = EXCLUDED._cdc_lsn
WHERE EXCLUDED._cdc_lsn > users_current._cdc_lsn;
--      ↑ 이미 저장된 것보다 더 최신(LSN이 큰)일 때만 업데이트
```

실행 결과:

- 이벤트 B (lsn=200) 먼저 도착 → 저장 (O)
- 이벤트 A (lsn=100) 나중 도착 → `100 > 200` = false → 무시 (O)

같은 이벤트가 두 번 와도 결과 동일 — 이를 ****멱등성(Idempotency)****이라 한다

UPSERT 일관성 — Kafka 파티셔닝의 역할

Kafka 레벨에서도 순서를 보장하는 메커니즘이 있다:

Debezium은 테이블의 PK를 Kafka Partition Key로 사용

user_id=42의 모든 이벤트 → Partition 3 (파티션 내 순서 보장)

user_id=99의 모든 이벤트 → Partition 7 (파티션 내 순서 보장)

- 같은 PK의 이벤트 → 같은 파티션에 순서대로 적재
- 단일 컨슈머가 파티션 하나를 처리하면 순서 뒤집힘이 거의 없음

Conditional UPSERT는 컨슈머 스케일아웃이나
장애 복구 시 리플레이를 대비한 안전장치

Consumer 코드 — 실제 구조

```
consumer = KafkaConsumer('dbserver.mydb.users')

while True:
    records = consumer.poll(timeout_ms=1000) # 메시지 없으면 블로킹 대기
    for record in records:
        event = record.value

        if event['op'] in ('c', 'u'): # INSERT or UPDATE
            upsert_to_current(event['after'])
            append_to_history(event['before'], event['after'])
        elif event['op'] == 'd': # DELETE
            delete_from_current(event['before']['user_id'])
            append_to_history(event['before'], None)

    consumer.commit() # "여기까지 처리했다" → 장애 시 이 지점부터 재개
```

`poll()` 은 busy waiting이 아닌 블로킹 호출 — CPU 낭비 없음

offset과 delivery 보장

Consumer가 Kafka에 "어디까지 읽었는지" 기록하는 것이 offset commit

commit을 처리 전에 하면?

→ 처리 중 장애 시 이벤트 유실 (at-most-once)

commit을 처리 후에 하면?

→ 처리 후 commit 전 장애 시 재처리 발생 (at-least-once)

실무 선택: at-least-once + 멍등성

- commit은 처리 후에 수행 (이벤트 유실 방지)
- 중복 처리는 **Conditional UPSERT (LSN 비교)**로 방어

"한 번 이상 전달, 하지만 결과는 정확히 한 번" = Effectively Once

4. CDC의 활용 범위

분석만을 위한 기술이 아니다

CDC의 다양한 Use Cases

① 분석 / 이력 추적

과거 데이터 변경 이력을 Warehouse에 쌓아서 분석
"고객 등급이 언제 변경됐는지", "가격 변동 추이"

② 시스템 간 데이터 동기화

유저 프로필 변경 → Elasticsearch 인덱스 갱신, 추천 시스템 피처 갱신

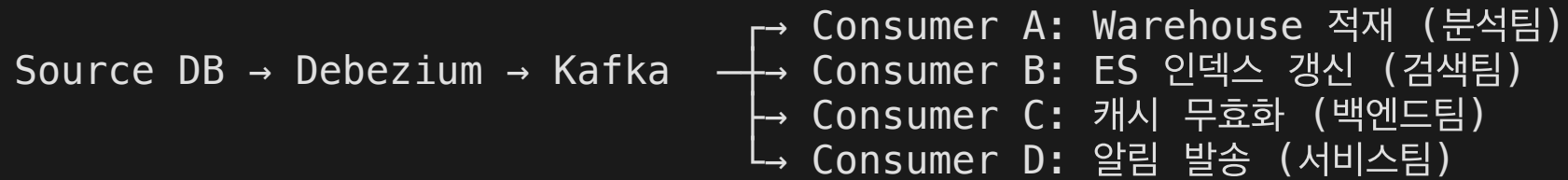
③ 마이크로서비스 간 데이터 전파

주문 상태 변경 → 배송 서비스, 정산 서비스에 자동 전달

④ 캐시 무효화

DB 변경 → 정확히 바뀐 키만 Redis에서 무효화

CDC 이벤트의 구독 모델



- 각 Consumer가 독립적인 consumer group으로 자기 offset 관리
- 서로 영향 없이 자기 목적에 맞게 이벤트 처리
- 새로운 Consumer 추가 시 기존 시스템 변경 불필요 (느슨한 결합)

배치 파이프라인과의 비교

	배치 방식 (dbt full-refresh)	CDC 스트리밍 방식
데이터 수집	주기적으로 <code>SELECT *</code>	변경분만 실시간 캡처
Source DB 부하	클 수 있음 (full scan)	거의 없음
Freshness	스케줄 주기 (1시간? 하루?)	초 ~ 분 단위
이력 추적	full-refresh → 이전 상태 유실	before/after 모두 보존
인프라 복잡도	낮음 (dbt + cron)	높음 (Kafka, Debezium 등)

실무에서는 — 섞어 쓴다

```
[CDC로 가져올 테이블]      주문, 결제, 유저 — 변경 잦고, 실시간성 중요, 대용량
    ↓
Kafka → raw layer
    ↗
[배치로 가져올 테이블]      dbt transformation → staging → mart
    ↓
dbt full-refresh → raw layer
코드 테이블, 설정값 — 거의 안 바뀜, 작은 테이블
```

CDC는 dbt를 대체하지 않는다

CDC = Ingestion(수집) 단계 / dbt = Transformation(변환) 단계

언제 CDC를 도입하는가?

CDC 도입을 고려해야 하는 시점:

조건	배치	CDC
테이블 크기가 작고, 분석 주기가 하루 이상	O	과잉
수억 행 테이블, 매번 full scan 비현실적	비효율	O
분 단위 이하 freshness 요구	불가능	O
Source DB에 부하를 줄 수 없음 (프로덕션 보호)	위험	O
변경 이력 (before/after) 보존 필요	유실 가능	O

복잡한 인프라 ≠ 좋은 인프라. 비즈니스 요구사항에 맞는 선택이 중요.

5. 애플리케이션 이벤트와의 차이

CDC 이벤트 vs. 백엔드에서 직접 발행하는 이벤트

백엔드 개발자 관점 — CDC는 비침투적

Log-based CDC에서 백엔드 개발자는 CDC의 존재를 몰라도 된다:

```
# 백엔드 개발자의 코드 — 평소와 완전히 동일
user = User.objects.get(id=42)
user.email = "new@x.com"
user.save()

# 이 아래는 백엔드 개발자가 전혀 신경 쓰지 않는 영역
# → DB에 UPDATE 커밋
# → WAL에 자동 기록
# → Debezium이 알아서 캡처
# → Kafka로 이벤트 전달
```

비침투적(non-invasive): 기존 코드 변경 없이 변경 이벤트를 캡처
백엔드 팀에 아무것도 요구하지 않아도 됨

CDC 이벤트 vs. 애플리케이션 이벤트

	CDC 이벤트	애플리케이션 이벤트
생성 주체	Debezium (DB 로그)	백엔드 코드 (직접 발행)
백엔드 코드 변경	불필요	필요 (producer 로직 추가)
이벤트 내용	row 레벨의 데이터 변경	비즈니스 의미가 담긴 이벤트
예시	email 필드가 a→b로 변경	사용자가 이메일을 변경함

핵심 차이

- CDC: "무엇이(What) 바뀌었는가"
- 애플리케이션 이벤트: "왜(Why) 바뀌었는가"

"관리자가 강제 변경한 건지, 유저가 직접 변경한 건지"는 CDC로 알 수 없음

애플리케이션 이벤트 — 코드 예시

```
def complete_order(order_id):  
    order = Order.objects.get(id=order_id)  
    order.status = "COMPLETED"  
    order.save()  
  
    # 비즈니스 의미가 담긴 이벤트를 직접 발행  
    kafka.produce("order-events", {  
        "event_type": "ORDER_COMPLETED",  
        "order_id": order_id,  
        "user_id": order.user_id,  
        "shipping_address": order.address,  
        "items": order.items,  
        "completed_by": "user"      # 유저 직접 완료  
    })
```

event_type, completed_by 같은 비즈니스 컨텍스트는
CDC 이벤트(row 변경)에서는 알 수 없는 정보

실무에서의 역할 분담

목적	적합한 방식
Warehouse 동기화, 데이터 복제	CDC
시스템 간 데이터 동기화 (ES, 캐시)	CDC
비즈니스 로직 트리거 (결제 완료 → 알림 발송)	애플리케이션 이벤트
비즈니스 감사 로그 (누가, 왜 변경했는지)	애플리케이션 이벤트

둘은 경쟁 관계가 아니라 역할이 다르다

하나의 시스템에서 두 가지를 함께 사용하는 경우가 많음

6. Dual Write Problem

그리고 Transactional Outbox Pattern

Dual Write Problem

앞선 애플리케이션 이벤트 발행 코드를 다시 보자:

```
def complete_order(order_id):  
    order = Order.objects.get(id=order_id)  
    order.status = "COMPLETED"  
    order.save() # ① DB 커밋  
  
    kafka.produce("order-events", { # ② Kafka 발행  
        "event_type": "ORDER_COMPLETED",  
        ...  
    })
```

만약 ①은 성공했는데 ②에서 Kafka가 죽어있다면?

- DB에는 주문 완료로 기록 (O)
- 배송팀은 이벤트를 못 받음 (X) → 데이터 불일치 발생

왜 해결하기 어려운가?

DB 커밋과 Kafka 발행은 서로 다른 시스템

하나의 트랜잭션으로 묶을 수 없다
→ 둘 중 하나만 성공하는 상황이 반드시 발생할 수 있다

문제의 본질

두 개의 서로 다른 시스템에 동시에 쓰는 것 = Dual Write

해결 아이디어

쓰기를 하나로 줄이자.

Kafka에 직접 쓰지 말고, DB에만 쓰자.

DB에 쓴 걸 이벤트로 바꾸는 메커니즘? → CDC

Transactional Outbox Pattern

이벤트를 Kafka에 직접 쓰지 않고, DB 테이블에 먼저 저장한다:

```
def complete_order(order_id):  
    with db.transaction(): # 하나의 트랜잭션  
        order = Order.objects.get(id=order_id)  
        order.status = "COMPLETED"  
        order.save() # ① 비즈니스 데이터  
  
        OutboxEvent.create( # ② 이벤트도 같은 TX에 저장  
            event_type="ORDER_COMPLETED",  
            payload={  
                "order_id": order_id,  
                "user_id": order.user_id,  
                "shipping_address": order.address  
            }  
        )  
    # ①②가 둘 다 성공하거나 둘 다 실패 - 트랜잭션 보장
```

Outbox 테이블 구조

```
CREATE TABLE outbox_events (  
  id BIGSERIAL PRIMARY KEY,  
  event_type VARCHAR(100) NOT NULL,      -- 'ORDER_COMPLETED'  
  aggregate VARCHAR(100) NOT NULL,      -- 'order'  
  payload JSONB NOT NULL,               -- 비즈니스 데이터  
  created_at TIMESTAMP DEFAULT now()  
);
```

- 비즈니스 로직과 같은 DB 트랜잭션으로 INSERT
- 이 테이블에 Debezium이 CDC를 걸면 자동으로 Kafka로 전달
- Kafka에 전달된 후 오래된 행은 주기적으로 정리(cleanup)

Outbox Pattern — 전체 흐름

[비즈니스 로직]

[인프라 레이어]



쓰기가 DB 하나로 줄어들음 → Dual Write Problem 해소

여기서 배운 모든 것이 연결된다

개념	Outbox Pattern에서의 역할
DB 트랜잭션	비즈니스 데이터 + 이벤트를 원자적으로 저장
CDC (Debezium)	Outbox 테이블의 변경을 캡처 → Kafka 전달
애플리케이션 이벤트	Outbox에 비즈니스 의미가 담긴 이벤트 저장
Kafka	이벤트를 여러 Consumer에 안전하게 분배
Consumer + offset	각자의 속도로 이벤트를 소비

CDC와 애플리케이션 이벤트가 경쟁이 아닌 협력 관계가 되는 패턴

7. 정리

하나의 원칙으로 모든 것이 연결된다

핵심 원칙

"두 시스템에 동시에 쓰지 마라.
하나에 쓰고, 나머지는 그걸 기반으로 파생시켜라."

이 원칙의 구현체들:

- CDC — DB에 쓰면 변경이 자동으로 이벤트가 된다
- Outbox — 이벤트도 DB에 쓰고, CDC로 전달한다
- Consumer — 각자의 목적에 맞게 이벤트를 구독한다

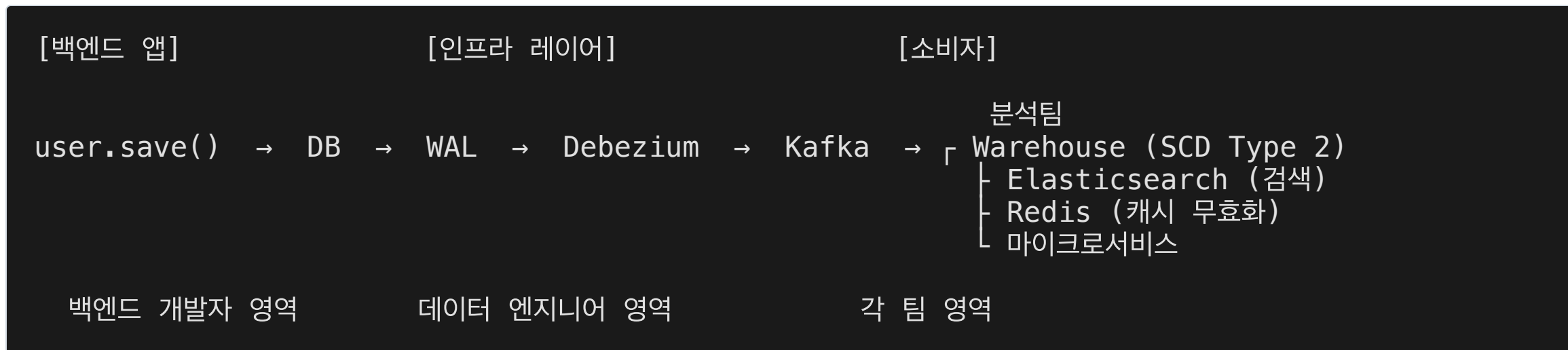
이 패턴은 DB + Kafka뿐 아니라, 분산 시스템 전반에서 반복적으로 등장한다.

의사결정 가이드

"데이터를 다른 시스템에 전파해야 한다"

- 실시간성 불필요 + 작은 테이블
→ dbt full-refresh (배치)
- 실시간 + row 레벨 변경 전파
→ CDC (Debezium + Kafka)
- 비즈니스 의미가 담긴 이벤트 전파
→ 애플리케이션 이벤트 발행
- 비즈니스 이벤트 + 신뢰성 보장 필요
→ Transactional Outbox + CDC

전체 파이프라인 — 최종 정리



각 영역이 독립적으로 동작하면서
CDC를 통해 하나의 Source of Truth로 연결된다

Q&A

궁금한 점이 있으면 질문해주세요